04

모니터링 체계 구축

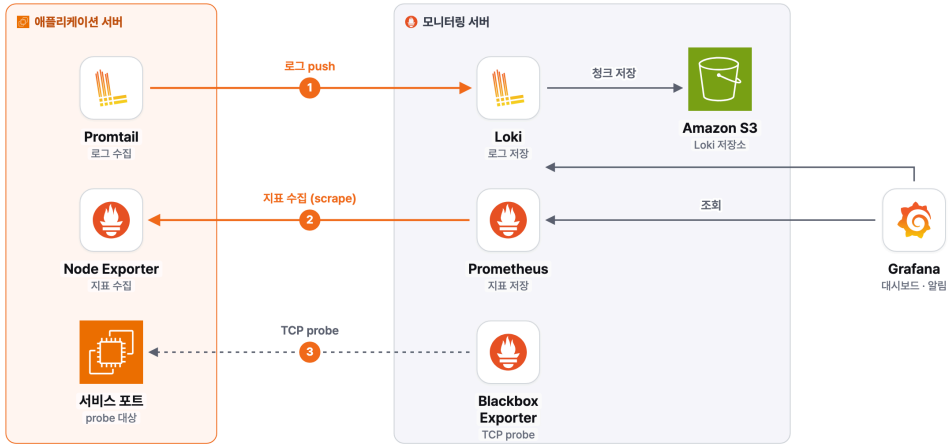
자체 제작 사내 관제를 Prometheus, Grafana, Loki로 교체하고 운영 서버 30대에 적용

역할	대안 검증, 설계, 운영 서버 30대 적용, 알림 체계 설계
환경	Prometheus, Grafana, Loki, AWS SSM
결과	운영 서버 30대 일괄 적용 서비스 DOWN을 앱 포트 40여 개 단위로 직접 확인 서버당 수집 에이전트 메모리 약 80MB, 지표 수집 주기 15초(기존 60초 폴링)

구성과 수집 방식

기존 관제의 한계

- 서버 1대의 자체 제작 시스템, 디스크 사용률 92%
- 서비스 DOWN은 "수집 실패" 알람으로만 간접 확인
- 시험한 상용 APM이 서버 8대에서 OOM Kill을 일으켜 철수



- 모니터링 서버는 공인 IP 없는 사설 서브넷, Docker Compose 4개 컨테이너에 CPU, 메모리 상한 지정
- Loki 저장소는 S3, 접근은 키 없이 인스턴스 프로파일. 수집 포트는 모니터링 서버 보안 그룹만 허용

수집 에이전트 비교 (실측)

구성	메모리	판단
Node Exporter + Promtail	약 80MB	**채택**
Grafana Alloy (통합)	235~255MB	제외

운영 서버 다수가 메모리 1GB급이고 대부분 Swap이 없어 소형 인스턴스 부담이 적은 쪽을 선택. 관리 대상 2개로 늘지만 장애 위험이 낮음. 모니터링 스택도 실측해 필요 메모리를 약 5GB로 산정.

서비스 DOWN 감지

- 운영 서버 30대의 앱 포트 40여 개를 Blackbox Exporter로 직접 TCP 확인. `probe_success == 0` 이면 DOWN
- 알림에 서버와 서비스 라벨을 붙여 어느 서버의 어떤 앱인지 즉시 식별
- 대상 목록은 리스닝 포트, 프로세스 작업 디렉터리, 프로세스 관리 도구 상태, 인프라 문서로 교차 확인

알림과 배포 자동화

기존 알림 기준 이전

레거시 관제의 이벤트 13종, 임계치, 채널, 발송 시간대를 조사해 Grafana 규칙과 1:1 매핑표 작성 후 전환. 임계치는 3단계에서 2단계로 축소.

분류	조건	수준
리소스	CPU, 메모리, 디스크 각 90% 초과	critical
에러 로그	5분간 에러 로그 1건 이상	warning
서비스 DOWN	앱 포트 TCP 확인 실패	critical
RDS	CPU 70% / 90% 초과	warning / critical

알림 피로 관리

- 복구 알림 필요 여부로 Contact Point를 나눠 type 라벨 4종으로 라우팅
- 에러 로그 알림은 10분 반복 억제, 복구 알림 없음
- 서버 이름, 현재 값, 해당 서버 대시보드 링크 포함

배포와 자동화

- AWS SSM Run Command로 운영 30대에 일괄 설치. 설치, 서비스 등록, 설정, 활성화, 확인 순서 고정
- Promtail 시작 전에 오프셋 파일을 미리 기록해 과거 로그 전체 재수집 방지
- 서버별 로그 대시보드를 Terraform으로 생성. Loki API의 서버 목록을 `for_each`로 받고, UID 고정과 `overwrite = false`로 기존 대시보드 보호
- 새 알림은 테스트 채널에서 검증 후 운영 채널로 이동. 테스트 로그 주입부터 수신까지 확인

이전과 이후

항목	이전	이후
서비스 DOWN	수집 실패로 간접 확인	**앱 포트 단위 직접 확인**
에러 감지	앱이 직접 전송	**Loki 쿼리 기반**
대시보드 추가	화면에서 수동 복제	**Terraform 자동 생성**